

Conversational AI Agent for a Streaming platform design document

- Author — Petr Janousek.
- Version — v1 - draft
- Status — Proposal / for review

Context

This document proposes a conversational assist layer inside the existing Search bar experience of a Netflix-like czech streaming platform. Users express vague, multi constraint or kind of cultural intent in Czech (“Something cozy for tonight”). The system returns structured reply: reply text, title picks from a catalog and tap chips for suggested continuation. That without being a standalone chatbot and without disrupting the playback flow.

Problem framing

The hard problem is not wiring the “LLM to search”, it’s the infrastructure around it and making and maintaining the model costs reasonable and with no hallucinations.

- Architectural decisions are optimised towards model costs being low and leaving no space for the model to hallucinate.
- Reliably and correctly query the catalog based on user intents and restrictions
- LLM hallucinating the available catalog products
- Reliably search by the czech language queries.

Proposed architecture

The very high level step-by-step proposed is:

1. Understand intent + extract and update constraints
2. Retrieve candidates from catalog index
3. Rank the retrieved candidates
4. Assemble the reply text, suggested movies/shows and chips reply

The LLM in this case is only extractor of intent and one sentence response writer, not the recommender itself. So every hard guarantee we have to enforce (choosing candidates, availability, constraints) is done deterministically, not by prompting.

End-to-end single turn

On user sent request:

1. Ingress and session load - Request hits the FastAPI backend; the service resolves *session_id* and loads the session object from Redis (state + capped history).
2. Guard - obvious abuse / prompt injection goes straight to refusal path and never reaches the model and other parts of the pipeline.
3. Extraction (LLM call) - Turns the user message into a state delta *{filters: [genre,..], constraints: [Czech only,..]}* and a semantic query.
4. Cache check - Normalised query checks Redis. On cache hit we skip straight to assemble step - cost reduction
5. State merge - We merge the state delta with the loaded session state — this is how the constraints actually stick. State is typed server-side object, not something the model remembers.
6. Hybrid retrieval - Retrieval over docs. Semantic and lexical search query over the same Elasticsearch index with hard constraint filters (origin, era,..). Both lists are fused. No hallucinations are possible, LLM is not used at all in this step.
7. Availability - The fused list of candidates is checked with Catalog Service that has real-time availability access. If title is not available, it is removed from the list.

8. Response message generation - Model gets the numbered list of candidates, picks (2-4) candidates from the list, writes the Czech reply text and selects chip templates from the phrase bank.
9. Validate and assemble - Assembles the response json — the reply text, list of chosen candidates' IDs and the chips. Validates the JSON schema.
10. Writing back - Write back the updated state and append turn to history in Redis. On failed generation/validation, we return at least the ranked candidates and template reply. We could also stream the message so the user is seeing responsiveness before the full message is assembled.

Catalog grounding

- Data
 - Live catalog service - platform's main DB where all the titles and availability lives - this is read-only source-of-truth for us and we don't want to "experiment" on the main DB.
 - We enrich the raw data from the catalog service with a single pass LLM call which adds metadata like tags, era, who's it for. Basically no cost and it lives in our DB.
- Indexing
 - Offline path - Embedder and indexer creates Elasticsearch index from our DB with enriched data. Each document is holding both the text fields (for lexical search BM25) and vector (for the vector search + kNN). Reindex nightly/on new catalog addition.
- Retrieval
 - Hybrid BM25 and kNN in one query with filters, which is then fused by RRF. — This is the main way to restrict hallucinations, LLM only chooses candidates from this fused list.
- Freshness
 - The freshness itself isn't strictly built into index (updated nightly so there can be stale data). The availability is checked against the source-of-truth platform's catalog service after the retrieval is done.

Multi-turn context and sticky constraints

- State is saved in Redis keyed by conversation ID. It is an object encapsulating the constraints and the turn history.
 - The constraints are updated only by the state delta's from the extractor and are valid until explicitly changed by the user request. The LLM never updates the state itself, it only creates state delta which is then deterministically merged — which makes the constraints reliable and sticky.
 - Thus the full history is never sent to the model — per turn token cost is basically flat even on the n-th step.

How is the structured output produced reliably

- Enforcement - JSON-schema output and validator. On any schema or other failure, we switch to the fail-path.
- candidates and chips picks are both from given as a list. The model then only chooses from these options, never generates them.

Where the LLM fits

- Intent and constraint extraction - This is where it fits using the LLM perfectly due to the natural-language understanding. It extracts the intent and constraints from the user's text.
- Choosing candidate titles - This is done by the retrieval part so we know that the result is correct, fresh and without any hallucinations.
- Generating reply text and choosing chips - This is again done by the LLM due to the language understanding. It generates a short one sentence reply with the found candidate titles and chooses correct chips for the conversation continuation that is relevant to the conversation.
- Validation - Must be deterministically correct schema of the returned object, we can't accept any hallucinations or other "thoughts" of the model, we don't use this in this step.

Technology choices

- Serving - FastAPI, Stateless, async orchestration. A single FastAPI process should handle ~150-250 turns/sec which should be enough with mentioned target load. We run several due for redundancy, not throughput.
- Session store - Redis - fast reads, free TTL, for session state and response cache. Also probably already used in stack
- LLM

- This one is a bit of open question and should be evaluated based on cost, precision on czech language. Before moving into production, there should also be eval for locally run models that would mitigate the costs per turn.
- intent pass - I'd start with smaller model of claude/openAI that probably have the best czech out of the alternatives.
- response pass - Start with smaller model, if not enough, use smarter model. for mvp again claude/openai models.
- Retrieval layer - Elastic search for both lexical and vector search.
- Vector store - vectors are stored in elasticSearch
- DB - PostgresDB — probably already in stack.
- chips templates - These are human-generated czech phrases for the chips, they live in DB and are cached in redis.

MVP scope cut

- Text only inside the existing search bar.
- Turns with sticky constraints
- Offline enrichment + nightly reindex - This is the “intelligence” behind the recommendations
- Availability filter against the Catalog Service
- Degradation path - Always return some response, never error.
- Guard - small first for injection prompting
- Hybrid retrieval
- Phrase bank with atleast few templates
- LLM gateway
- Evaluation set and ability to rate the response
- Sized for 5-10% of the traffic targeted. (Even on full launch I dont think every subscriber uses this.)

Deferred:

- personalisation and history - Whole other scope that I'd probably defer from on first production run as well
- any cross-session history - Similar to the point above, this would make the scope much larger
- kids/ other types of profiles - No kids gate for horrors for example
- voice - Another processing addition of voice in this case
- chips are generated by llm, not chosen from templates - probably need evaluation, templates are simpler for first version.

Top risks

- LLM costs at scale and possible rate limits - Costs definitely can skyrocket — Heavy caching + possible evaluation of locally run models.
- prompt injections - These can have severe consequences — the guard before the llm intent call needs to be through roughly tested even though we just the received message only as a data, not prompt.
- Quality of Czech responses — A stated quality gate, but I've deliberately minimized the risk by keeping generated Czech to 1–2 sentences and using templated chips. The larger Czech risk is on the search side (morphology, vague intent), which lemmatization + enrichment + a Czech-capable embedding model address. Mitigation is a human-rated Czech eval set; if beta evals pass, I don't consider this a launch blocker.
- Very vague intent - “something nice” is too vague for the searches. Probably some defined paths should be created to return some “popular title candidates right this week” for such intents. This can also be partly mitigated by the later personalisation from which we could derive more specific candidates.
- Retrieval relevance — If enrichment or the Czech embedding is weak, “spy film with the older guy from the 90s” returns junk and no reply text saves it. Mitigation: enrichment quality is the highest-leverage spend; pick the embedding model on measured Czech retrieval accuracy; track retrieval hit-rate on the eval set as a first-class metric, separate from reply quality.